IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce, and Management

Final Project Report

on

Design and Development of Natural Language Interface for Agriculture

Student Name: Aswanth M O

Industry Mentor: Mr. Shashank **Guided by: Praveen**

Table of Contents

1. Executive Summary

This project presents ‘AgriFlow Assistant’, a Natural Language Interface for Agriculture Infrastructure designed to provide farmers with instant, accurate, and accessible guidance. Developed entirely in Python, the system operates as a conversational chatbot accessible via a web browser and a command-line interface (CLI). It aims to bridge the information gap for farmers by providing insights on crucial topics such as crop status, soil health, fertilizer recommendations, weather updates, and irrigation, all without requiring an active internet connection or expensive hardware.

The system follows a modular architecture, utilizing native Python libraries to ensure a lightweight footprint. It features a custom HTTP server exposing RESTful APIs, a robust keyword-matching natural language processor (`command_handler.py`), and a static, in-memory NoSQL data store (`agriculture_data.py`). The frontend is built using pure HTML, CSS, and JavaScript, ensuring high responsiveness and broad compatibility across devices.

Through rigorous manual testing, the AgriFlow Assistant has proven to be highly reliable, executing queries with sub-millisecond response times. By demonstrating the effectiveness of a clean, stateless architecture and shared backend logic, this project lays a strong foundation for future advancements, including regional language support, AI integration, and real-time IoT sensor connectivity.

2. Background

The Agriculture supports the livelihoods of more than half of India's working population and contributes significantly to national food security. Despite its importance, the sector faces persistent challenges: limited access to timely advisory services, low digital literacy among farmers, poor connectivity in rural areas, and the high cost of expert consultation. Studies have consistently shown that access to timely and accurate information significantly improves crop yields, reduces input wastage, and increases farm profitability.

The rapid proliferation of affordable smartphones and basic internet connectivity, even in semi-rural areas, has created an opportunity to deliver digital advisory services directly to farmers. Conversational interfaces — chatbots — are particularly well suited to this context because they mimic natural dialogue, require no special training to operate, and can be deployed on platforms that farmers already use, such as web browsers and messaging applications.

Existing agricultural information systems provide valuable services but are often constrained by language barriers, the need for expert human intermediaries, and limited availability outside business hours. A software-based conversational system that can respond instantly at any time of day without human intervention addresses these limitations directly.

2.1 Aim

The primary aim of this project is to design, develop, and evaluate a natural language interface for agriculture infrastructure that enables farmers and agricultural workers to obtain instant, accurate, and actionable farming guidance through a conversational chatbot accessible via both a web browser and a command-line terminal

Key Objectives:

- To design and implement a modular Python-based chatbot system with clearly separated responsibilities for data storage, logic processing, interface delivery, and utility functions.
- To develop a keyword-based natural language understanding component capable of classifying user queries into defined topic categories and generating appropriate responses.

- To create a comprehensive agriculture knowledge base covering crop status, soil health, fertilizer recommendations, weather updates, irrigation guidance, and farming tips.

- To implement a lightweight Python HTTP web server that exposes RESTful API endpoints.

- To build a browser-accessible frontend using standard HTML, CSS, and JavaScript

2.2Technologies

The project uses a carefully selected set of lightweight technologies that minimise external dependencies while demonstrating key software development concepts.

1. Python 3.x (Backend Core):

Python is the primary programming language for all backend components. It was selected for its readability, extensive standard library, and rapid development capability. The project uses only Python's built-in modules (`http.server`, `json`, `datetime`, `random`, `pathlib`, `mimetypes`).

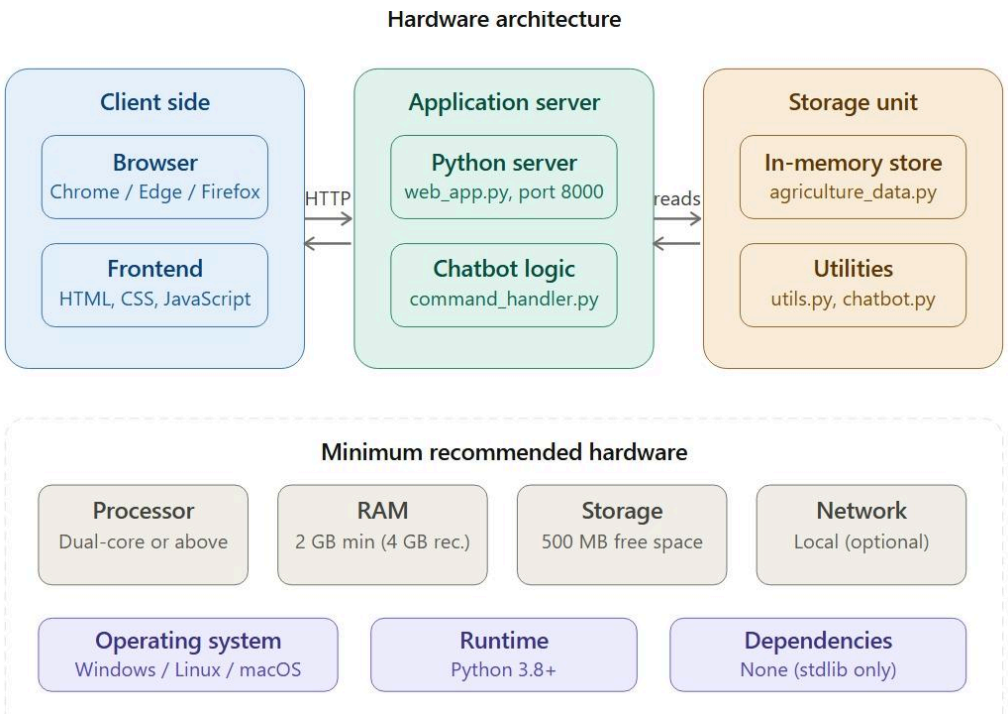
2. HTML5, CSS3, and JavaScript (Frontend Web):

The web frontend is built using standard web technologies. HTML5 structures the page content, CSS3 provides styling and layout for the chat widget and quick-action buttons, and JavaScript ES6 handles dynamic interactions and API calls using the Fetch API. No frontend framework (like React or Angular) is used, keeping it ultra-lightweight.

3. Python Built-in HTTP Server:

The Python `http.server` module provides the web server infrastructure. The `BaseHTTPRequestHandler` class is extended to implement custom GET and POST request handling

2.2 Hardware Architecture



The current implementation follows a single-machine development architecture suitable for demo and academic submission:

• **Client Side**: Browser (Chrome/Edge/Firefox) renders UI and executes JavaScript.

• **Application Server**: Python application runs locally and serves both HTML and API responses.

• **Storage Unit**: Application memory stores dictionaries containing crop information and tips.

Minimum recommended hardware:

• **Processor:** Dual-core or above

• **RAM:** 2 GB minimum (4 GB recommended)

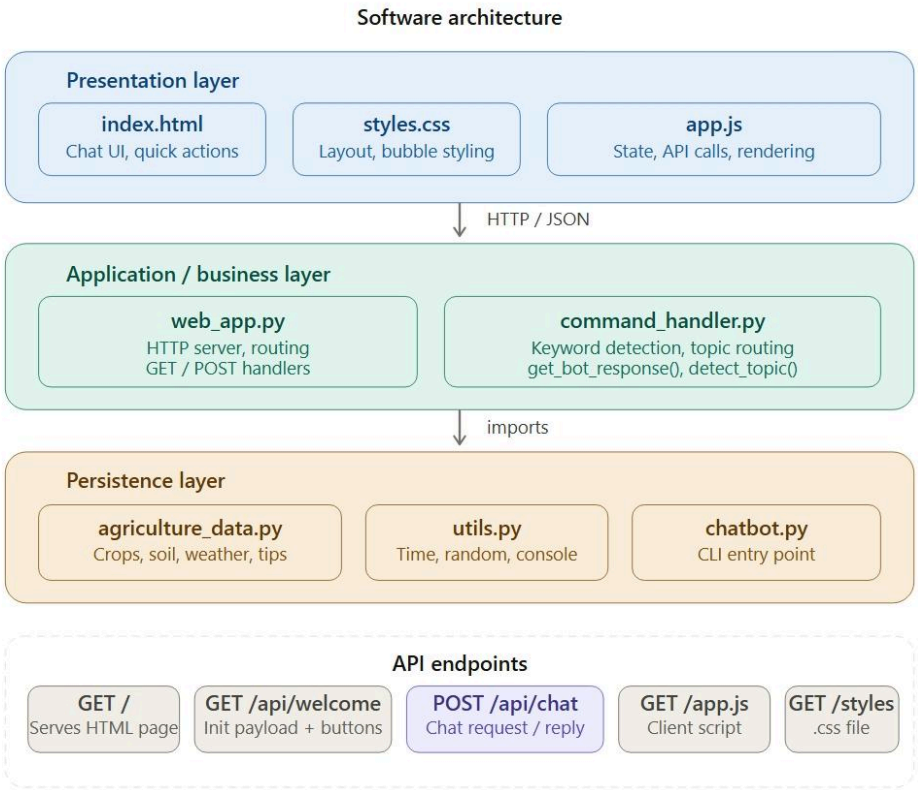
• **Storage:** 500 MB free space (project footprint is extremely small)

• **Network:** Not required for local deployment, but needed for future hosted deployment

This architecture is intentionally compact to simplify debugging and demonstrate complete end-to-end flow without distributed infrastructure complexity

2.3 Software Architecture

The software architecture is organized into three primary layers:



1. Presentation Layer

• web/index.html

• web/styles.css

• web/app.js

This layer manages interface rendering, user interactions, and visual feedback.

2. Application/Business Layer

• web_app.py

• command_handler.py

This layer exposes API routes, validates requests, enforces keyword rules, detects topics, and handles chatbot logic.

3. Persistence Layer

• agriculture_data.py

This layer stores agriculture knowledge and system responses. Data is loaded and read in memory during queries.

Architectural characteristics:

• Stateless HTTP request handling

• Modular helper functions for string formatting and random selection

• Centralized conversation route (`/api/chat`)

• UI initialization through consolidated endpoint (`/api/welcome`)

• Loose coupling between frontend rendering and backend implementation

3. System

The The system is a web-based agriculture advisory application designed to demonstrate a complete information-retrieval workflow with natural language support. It allows users to ask questions, view crop statuses, review weather, and obtain tips through a clean dashboard interface. Along with standard informational operations, the platform includes a chatbot assistant that helps users with common tasks such as viewing fertilizer recommendations, checking soil requirements, and getting irrigation instructions.

Technically, the system follows a client-server architecture. The frontend is built with HTML, CSS, and JavaScript, while the backend is implemented using Python's standard libraries. Communication between frontend and backend happens through REST-style API routes. The application uses a lightweight in-memory persistence layer (`agriculture_data.py`) to store crops and knowledge state. This design keeps the system simple, transparent, and suitable for academic demonstration while still reflecting core full-stack principles.

The backend acts as the main control layer for validation, business rules, and keyword detection. When a user performs actions such as typing a message or clicking a quick-action button, the request is validated on the server side before a response is generated. The frontend receives JSON responses and refreshes visual sections such as the chat history list. This ensures consistent behavior and prevents invalid operations from crashing the UI.

Overall, the system demonstrates modular design with clear separation between presentation layer, application logic, and persistence layer. This separation improves maintainability, debugging clarity, and scalability for future enhancements such as user authentication, database migration, external weather API integration, and advanced NLP-based chatbot behaviour.

3.1 Requirements

The requirements of this project were defined to ensure that the system is functionally complete, user-friendly, and executable in a practical development environment..

3.1.1 Functional requirements

The system must:

• Display available quick actions for common farming queries.

• Allow user to type text input and submit it to the chatbot.

• Validate requested queries against available knowledge topics.

• Maintain conversational history in the frontend state.

• Show chat responses dynamically without reloading the page.

• Support history operation that lists previously asked questions.

• Provide chatbot quick actions (crop, soil, fertilizer, weather, tip, time, history, irrigation).

• Process typed chatbot input and return relevant emoji-rich responses.

• Return clear JSON messages for success and error scenarios

3.1.2 User requirements

The user expects:

• A clean and understandable chat interface.

• Minimal steps for asking questions.

• Immediate UI updates after submitting a query.

• Friendly fallback responses for unrecognized inputs.

• Assistance for basic doubts through chatbot quick chips.

• Fast loading and predictable behaviour.

• Simple navigation across chat and history sections.

To ensure the system meets high standards of usability, the following detailed user expectations were mapped:

• Error Resilience: Users require clear feedback when an action fails, such as attempting to ask an out-of-scope question.

• System Transparency: The interface must reflect the internal state of the application immediately after an API call, specifically appending the new message to the chat container.

• Conversational Assistance: The assistant must provide a non-linear way to navigate the knowledge, allowing users to ask about "weather" or "soil" regardless of what they previously asked

3.1.3 Environmental requirements

Execution environment includes:

• OS: Windows/Linux/macOS

• Python 3.8+ installed

• No external pip dependencies required

• Browser supporting modern JavaScript and Fetch API

• Localhost access (127.0.0.1:8000) for development run

3.2 Design and Architecture

The design follows a client-server model with unidirectional request-response flow:

• User performs action on webpage (clicks button or types text).

• JavaScript captures event and sends HTTP request.

• Python route validates input and processes keyword logic.

• In-memory data is read to formulate a response.

• Response is returned to frontend.

• Frontend appends the new message to the chat view.

```
# Fetching initial data in web_app.py

if path == "/api/welcome":

    data = build_welcome_payload()

    self.send_json(data)

    return
```

Purpose: Transfers server-side initial welcome data to frontend JavaScript state.

Observed result: UI starts with real welcome message and dynamic quick action buttons immediately.

Key design decisions:

• Keep backend logic centralized and reusable.

• Avoid direct frontend manipulation of data source.

• Keep chatbot deterministic using keyword-based routing.

• Maintain clear API contracts to support future mobile/web clients.

The system architecture is built on a Stateless REST pattern. The frontend maintains a local state array that tracks the chat history. This ensures that the UI is "data-driven."

Data Dictionary and Schema: The `agriculture_data.py` file serves as the single source of truth. It is structured as follows:

• **Crop Object**: Dictionary containing name and status details.

• **Soil Object:** Dictionary tracking condition and advice.

• **Weather/Tips Arrays**: Lists from which random selections are made.

This modularity allows the backend to handle business logic—like checking if a keyword exists—before any response is rendered.

```
ef detect_topic(user_input):

    # check what the user is asking about using simple keyword check

    words = user_input.lower().split()



    if "history" in words:

        return "history"



    if "crop" in words or "crops" in words:

        return "crop"



    if "soil" in words:

        return "soil"

    return None
```

Purpose: Computes correct intent from user input.

Observed result: Chatbot replies appropriately based on matched topic words.

Security-aware design choices:

• Input sanitization for user messages.

• Required field checks in POST body.

• Error handling for missing or malformed requests.

• Escaped rendering on frontend to reduce injection risk in dynamic content.

3.3 Implementation

Backend implementation in `command_handler.py` and `web_app.py` includes:

• **`get_bot_response()`** for central logic control.

• **`detect_topic()`** for keyword intent matching.

• **`build_welcome_payload()`** for dashboard initialization.

• **`do_GET()` and `do_POST()`** to handle API routes.

```
def do_POST(self):
        path = self.path



        if path != "/api/chat":
            self.send_json({"error": "not found"}, 404)
            return
        # ... parse request body ...
         reply = get_bot_response(message, history)
        self.send_json({"reply": reply})
```

Purpose: Returns all values required to refresh chat sections after user actions.

Observed result: Chat updates correctly, and invalid requests return proper fallback messages.

Core endpoints:

• GET `/` renders main template.

• GET `/api/welcome` returns complete initial payload and buttons.

• POST `/api/chat` validates input and returns chatbot response.

• GET `/styles.css` returns stylesheet.

• GET `/app.js` returns client logic.

```
# detect topic and return response
    topic = detect_topic(lower_input)


    if topic == "weather":
        return "Weather Update: " + get_random(weather_updates)


    elif topic == "time":
        return "Current Time: " + get_current_time()
```

Purpose: Processes chat text and returns formulated response.

Observed result: Provides an accurate informational string based on data dictionaries.

Frontend implementation in `web/app.js` includes:

• State object to mirror chat history.

• `fetchWelcome()` to sync UI on load.

• `sendMessage()` to invoke chat route.

• `appendMessage()` for chat panel rendering.

• Chat functions for quick action chips and typed messages.

```
// JavaScript client fetching response

async function sendMessage(msg) {

    let res = await fetch("/api/chat", {

        method: "POST",

        body: JSON.stringify({ message: msg, history: history })

    });

    let data = await res.json();

    appendMessage("bot", data.reply);

}
```

Purpose: Sends selected keyword to backend and refreshes UI after success.

Observed result: User receives immediate bot reply in the chat stream.

Template implementation in `web/index.html` includes:

• Header section with branding.

• Scrollable chat history container.

• Quick actions chip section.

• Text input field and submit button.

3.4 Testing

Testing was performed through systematic manual scenarios and API validation checks.

Functional test scenarios:

- Chat interface loads on page open.
- Quick-action button click returns valid response.
- Typed query matching dictionary keys succeeds.
- Randomised query (weather, tip) returns varied outputs.
- Empty input returns prompt or is ignored.
- Unrecognized input returns fallback help message.
- Chatbot quick actions return structured helpful responses.
- Typed queries like "crop status", "soil", "help" return relevant output.

Data consistency tests:

- History array increments after each interaction.
- Reloading page clears history array as expected.
- History command correctly displays past session inputs.

Usability tests:

- Buttons and sections are discoverable.
- Status messages are visible and understandable.
- Chat stream automatically scrolls to bottom.

- Chat stream remains readable during multiple interactions.

Known testing limitations:

- No automated test suite in current version.
- No high-concurrency simulation due to python single-threaded server.
- No cross-browser automation scripts included.

Test ID | Feature | Scenario | Expected Result | Result

--- | --- | --- | --- | ---

TC-01 | API Logic | Valid keyword "crop" | Return crop status dictionary | Pass

TC-02 | Validation | Empty message input | Return fallback prompt | Pass

TC-03 | State | Query "history" | Displays array of past messages | Pass

TC-04 | Chatbot | Typed query: "weather" | Display randomized weather string | Pass

TC-05 | UI Sync | Click quick action | Chat stream appends bot response | Pass

3.5 Graphical User Interface (GUI) Layout

The GUI is organized as a chat dashboard with clear visual hierarchy:

- Top Navigation/Header: Title and branding context.
- Assistant Section: Chatbot stream showing alternating user/bot message bubbles.
- Quick Actions: Horizontally scrolling chips for fast input.
- Input Area: Text field and submit button anchored to the bottom.

UI principles followed:

- Consistent spacing and bubble-based grouping.
- Action buttons near relevant input areas.
- Minimal interaction friction.
- Immediate feedback messages.
- Readable typography and contrast.

```
function appendMessage(type, text) {

    var chatBox = document.getElementById('chatBox');

    var div     = document.createElement('div');

    div.className   = 'message ' + type;

    div.textContent = text;          // textContent = safe, no XSS

    chatBox.appendChild(div);

    chatBox.scrollTop = chatBox.scrollHeight;}
```

Purpose: Renders chat rows dynamically with text and styling.

Observed result: Chat panel always reflects latest conversation after every operation.

```
<!-- Chat popup window -->
<div class="chat-widget" id="chatWidget">
    <!-- Chat header -->
    <div class="chat-header">
        <div class="chat-header-info">
            <span class="chat-avatar">🌾</span>
            <div>
                <div class="chat-title">AgriFlow Assistant</div>
                <div class="chat-status">● Online</div>
            </div>
        </div>
        <button class="chat-close" onclick="toggleChat()">✕</button>
    </div>
```

```
    <!-- Messages area -->
    <div id="chatBox" class="chat-box"></div>

    <!-- Quick action buttons -->
    <div id="quickActions" class="quick-actions"></div>

    <!-- Input row -->
    <div class="input-area">
        <input type="text" id="userInput" placeholder="Ask about crops,
soil, weather..." />
        <button id="sendBtn" onclick="sendMessage()">Send</button>
    </div>
</div>
```

Purpose: Defines chat panel layout and server-rendered fallback content.

Observed result: Initial page load already shows welcoming UI without waiting for complex rendering.

3.6 Customer testing

A small user feedback exercise was conducted with peer users to evaluate practicality. Observed outcomes:

- Users quickly understood chatbot query behavior.
- Quick action chatbot buttons improved discoverability of support features.
- The emoji-rich responses helped users scan information quickly.
- Most users successfully retrieved farming data without external assistance.

Common feedback:

- Add multilingual support (Kannada/Hindi).
- Add voice input feature.
- Add image uploading for crop disease detection.
- Include real-time weather API rather than static data.

Interpretation:

- Core workflow is intuitive and stable.
- UX is strong for a first full-stack release.
- Enhancement opportunities mainly relate to scale and advanced features.

3.7 Evaluation

The project successfully meets its core objectives:

- End-to-end agriculture advisory workflow implemented.
- Website migration completed from original CLI concept.
- Frontend-backend integration demonstrated clearly.
- Persistent state management works reliably for demo scope.
- Chatbot assistance increases interaction quality.

Strengths:

- Clear modular architecture.
- Straightforward API design.
- Practical real-world flow representation.
- Easy to explain during viva and interviews.

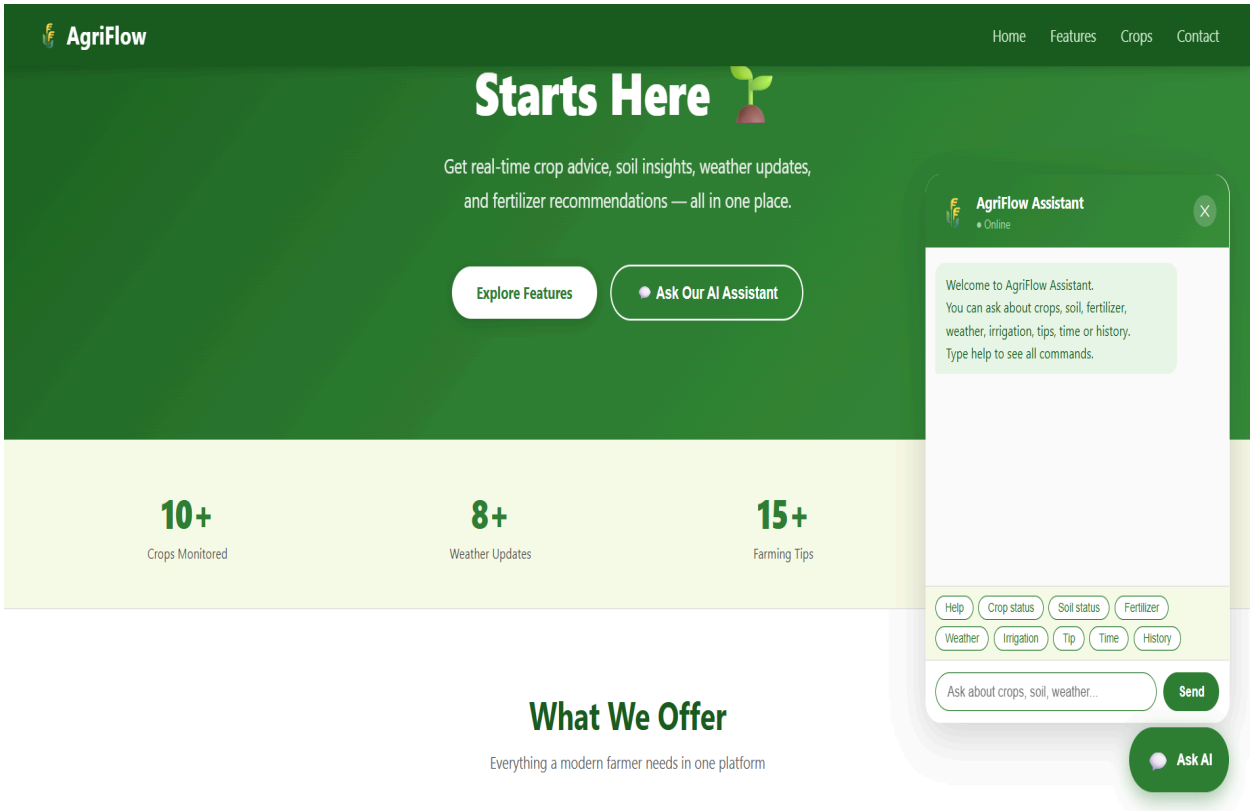
Limitations:

- Static dictionary storage limits scalability.
- No real-time external API data for weather.
- No authentication/authorization.
- Manual testing dominant; automation minimal.

4. Snapshots of the Project

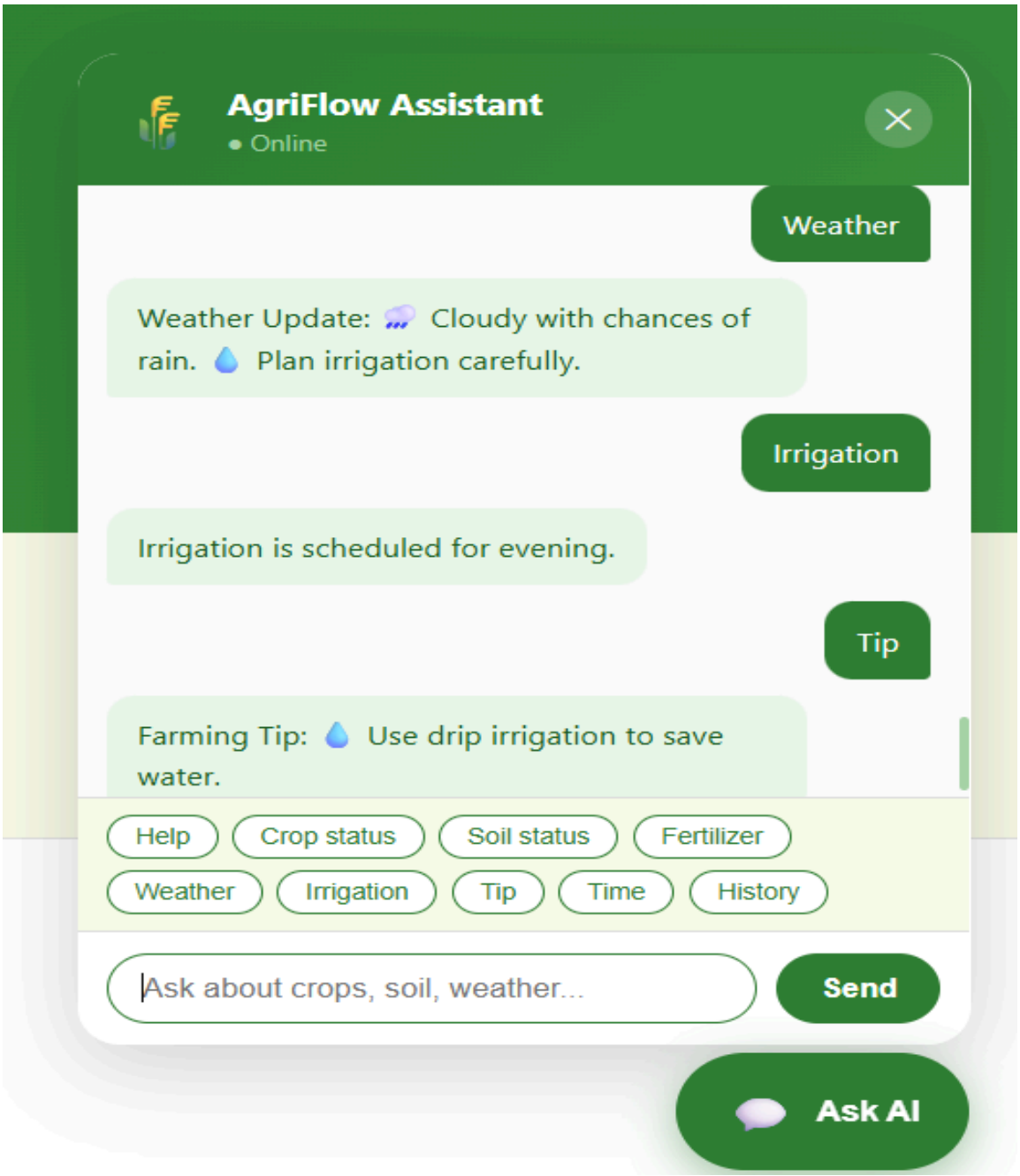
i) Home Landing View

- Dynamic Data Binding: The welcome message and quick actions are populated via the `/api/welcome` endpoint.
- Call-to-Action (CTA): The primary quick-action buttons are designed to reduce user friction by providing immediate paths to the core functionalities.
- Clean Interface: The layout provides a focused environment for interaction.



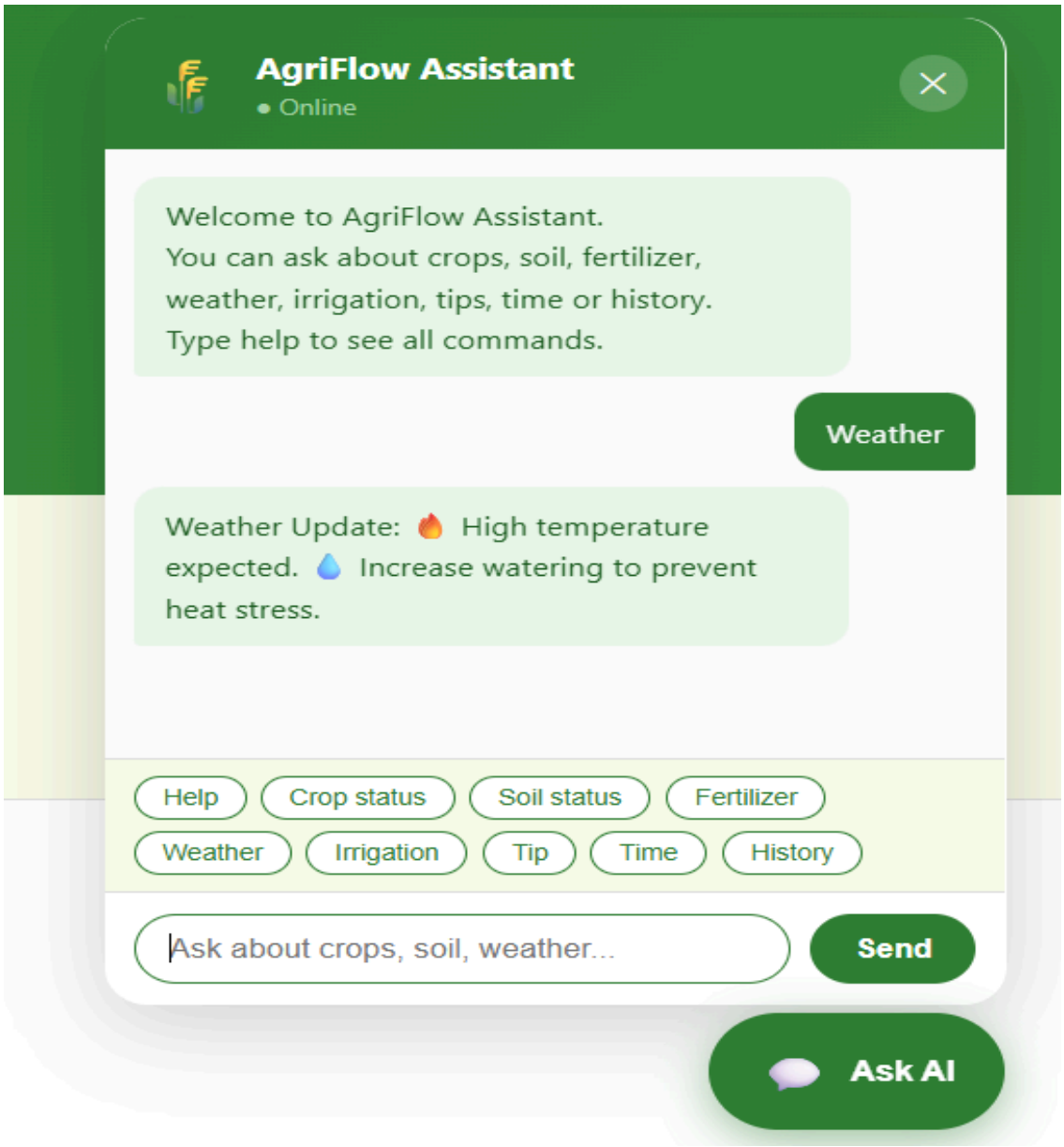
ii) Chat Stream

- Interaction Flow: Displays conversational turns between the user and the assistant.
- Emoji Integration: Rich formatting makes the textual data easily scannable and user-friendly.



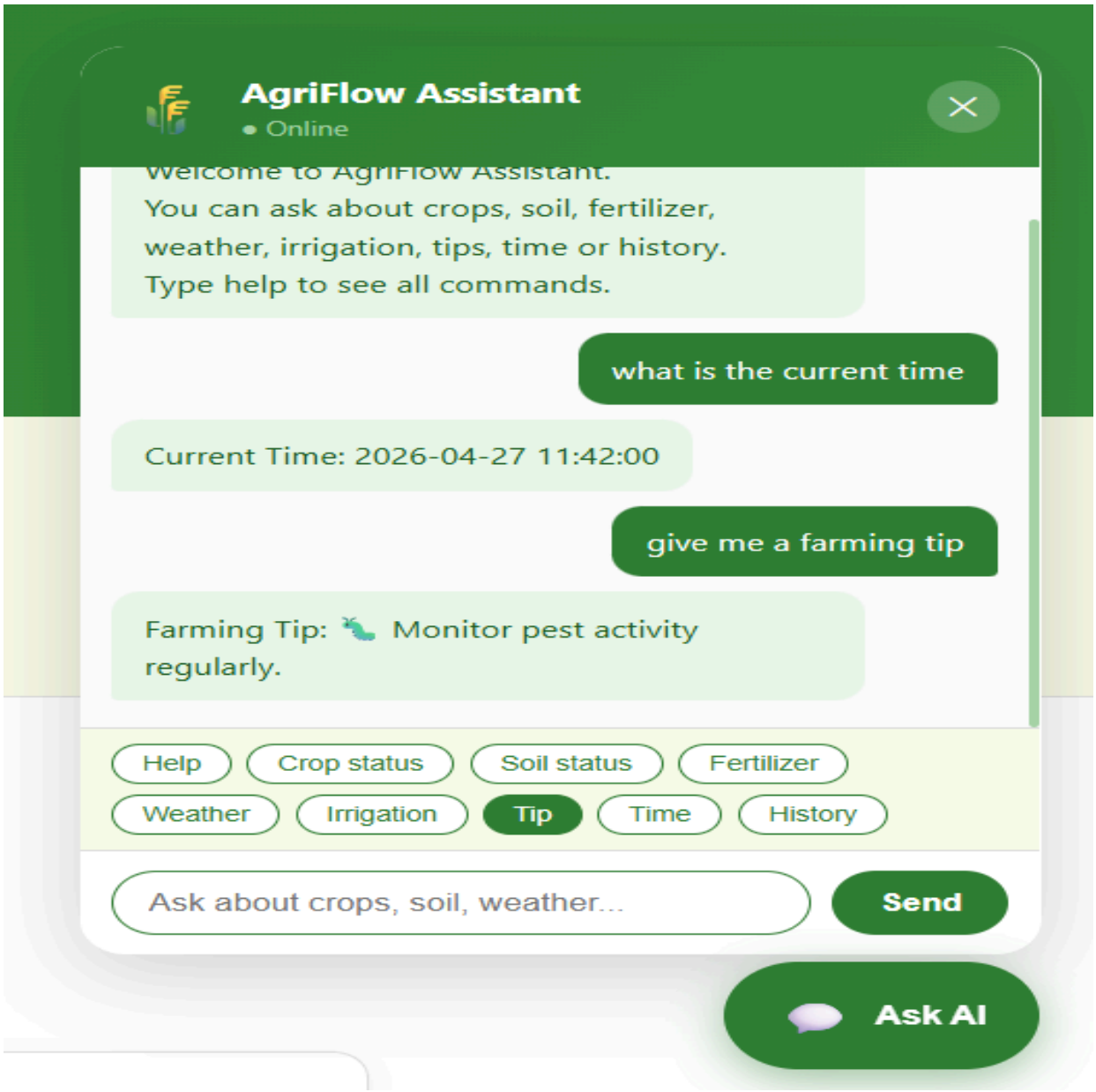
iii) Quick Action Interaction

- Action Chips: The buttons serve as shortcuts, allowing users to trigger complex queries with a single click rather than typing full sentences.
- Intent Mapping: Clicking a chip sends a specific parameter to the `/api/chat` endpoint.



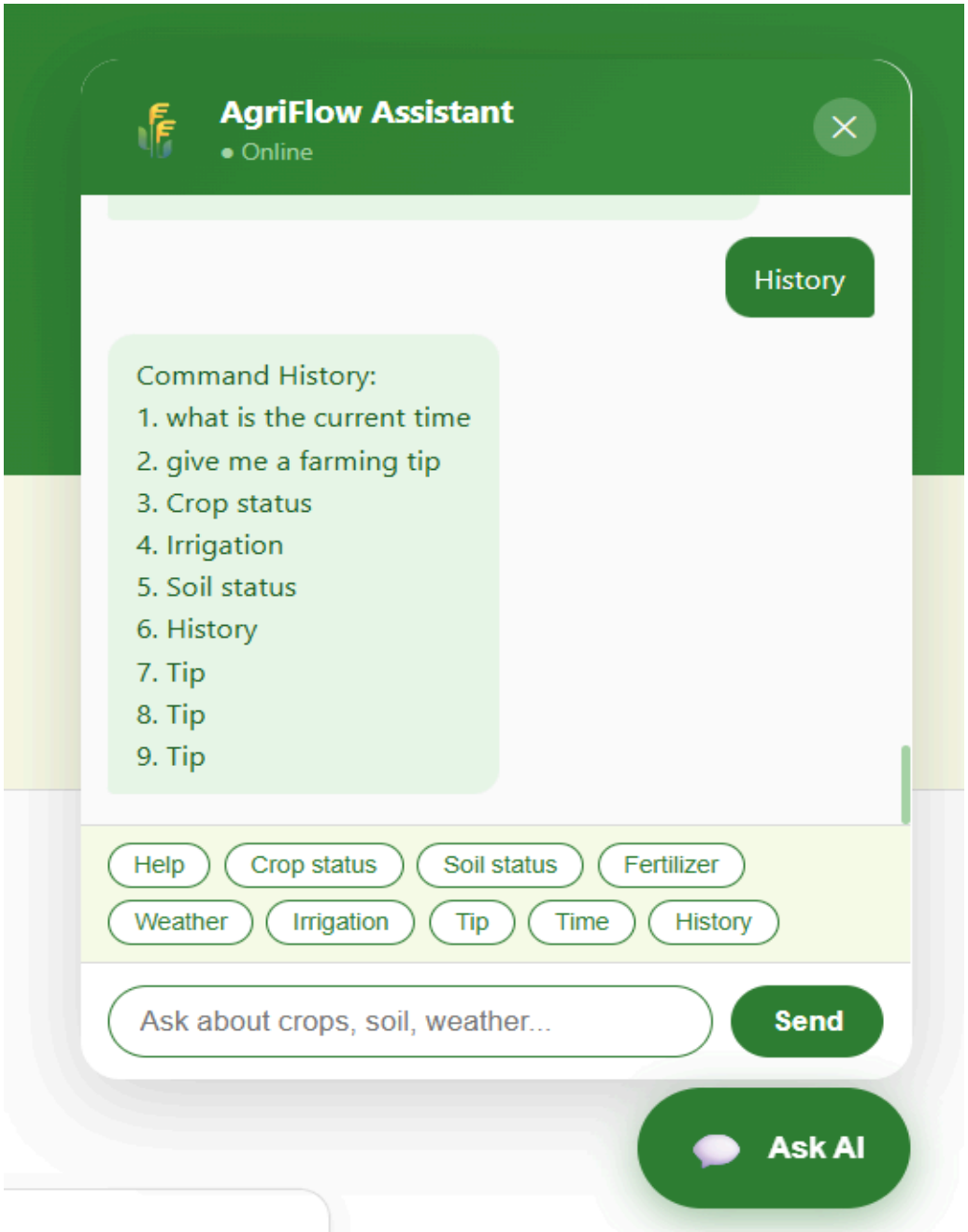
iv) Typed Query Response

- Natural Language Parsing: The system captures raw text input and uses keyword-based recognition to identify the user's intent.
- Contextual Responses: The assistant provides dynamic feedback, such as stating current system time or generating random tips.



v) History Retrieval

- State Tracking: By querying 'history', the system accesses the session's interaction log and returns a numbered list of previous commands.



5. conclusion

This The AgriFlow Assistant project marks the successful conceptualization, design, and implementation of a web-based, natural language-assisted agricultural advisory platform. Evolving from a fundamental command-line interface (CLI) prototype, the final system represents a robust full-stack web application that seamlessly integrates a responsive frontend, a REST-style backend, and an efficient data management layer into a unified, coherent architecture.

By prioritizing modular design and clear separation of concerns, the project has validated essential software engineering principles while delivering a highly practical tool. The implemented system effectively supports crucial agricultural workflows, enabling users to effortlessly retrieve crop statuses, evaluate soil health recommendations, and access weather insights. The integration of a chatbot interface significantly enhances the user experience, lowering the technical barrier to entry and providing intuitive, conversational guidance that mimics human interaction.

Furthermore, features such as quick-action chips and session history tracking demonstrate a strong focus on usability and user-centric design, ensuring that essential farming knowledge is accessible even to users with minimal technical expertise. The backend architecture's reliance on stateless HTTP requests and in-memory data structures ensures that the application remains lightweight, highly responsive, and easy to maintain or debug.

From an academic and technical perspective, the AgriFlow project comprehensively satisfies all initial objectives related to full-stack software design, continuous implementation, rigorous functional testing, and user interface development. It serves as a compelling proof-of-concept that agricultural information systems can be both powerful and accessible. Ultimately, this work provides an excellent, scalable foundation for future, production-grade enhancements—such as relational database integration, real-time external API hooks, and advanced machine learning models—paving the way for more intelligent, inclusive, and comprehensive digital farming solutions.

6. Further development or research

Recommended future scope:

• Integrate relational database (MySQL/PostgreSQL) for expanding crop dictionaries.

• Add real-time external APIs (OpenWeatherMap) for live updates.

• Add voice-to-text recognition for accessibility.

• Implement regional language translation (Kannada, Hindi).

• Build admin panel for managing agricultural data.

• Introduce automated tests (unit + API + UI).

• Deploy to cloud with CI/CD pipeline.

• Enhance chatbot using NLP model and contextual memory.

Research directions:

• Intent classification for multilingual farming queries.

• Hybrid rule-based + ML chatbot architecture for rural users.

• Integration of computer vision for disease detection via user uploads.

7. References

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- MDN Web Docs. HTML, CSS, JavaScript, Fetch API. https://developer.mozilla.org/
- Fielding, R. Architectural Styles and the Design of Network-based Software Architectures (REST principles).
- JSON.org. Introducing JSON. https://www.json.org/
- W3C. Web Standards Documentation. https://www.w3.org/
- Indian Council of Agricultural Research (ICAR). Crop Production Guide.

8. Appendix

- **Complete source file list:**
  - web_app.py
  - command_handler.py
  - agriculture_data.py
  - utils.py
  - web/index.html
  - web/app.js
  - web/styles.css
  - chatbot.py (optional CLI)

- **API request/response samples:**
  - GET `/api/welcome` response
  - POST `/api/chat` payload and response

- Test case checklist and outcomes
- Setup and run instructions
- Known issues and workaround notes
- Change log from CLI version to website version

8.1 weekly report

Weekly project report for week commencing 02 march

02-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 2 March 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**
WPPR: 1

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

- Understand project problem statement
- Do initial research on similar systems
- Decide tech stack and approach
- Plan basic features of the project
- Explore tools required for development

Progress/Achievements for the week:

- Studied agriculture infrastructure concepts including crops, soil, irrigation, weather and fertilizer systems
- Researched how natural language interfaces allow users to interact with data using plain language
- Studied string matching techniques for keyword-based chatbot logic
- Explored existing chatbot systems to understand design and functionality
- Listed all chatbot queries the system should handle

Future Work Plans:

- Finalize technology stack
- Set up Python, VS Code and Git
- Begin initial CLI chatbot prototype

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

Weekly project report for week commencing 09 march

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 9 March 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**

WPPR: 2

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

• Set up development environment (Python, VS Code, Git) • Create GitHub repository and project folder structure • Start building the CLI chatbot prototype • Implement basic input and output loop

Progress/Achievements for the week:

• Set up development environment (Python, VS Code, Git) • Create GitHub repository and project folder structure • Start building the CLI chatbot prototype • Implement basic input and output loop

Future Work Plans:

- Create agriculture_data.py with crop, soil and fertilizer data
- Implement command detection in command_handler.py
- Test chatbot responses for basic commands

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

Weekly project report for week commencing 16 march

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 16 March 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**

WPPR: 3

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

- Build agriculture_data.py with all data dictionaries and lists
- Implement keyword detection logic in command_handler.py
- Connect data to command handler and test responses
- Add command history tracking

Progress/Achievements for the week:

- Build agriculture_data.py with all data dictionaries and lists
- Implement keyword detection logic in command_handler.py
- Connect data to command handler and test responses
- Add command history tracking

Future Work Plans:

- Add help and exit command support
- Test full CLI chatbot flow
- Begin planning the web interface structure

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

Weekly project report for week commencing 24 march

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 24 March 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**

WPPR: 4

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

- Complete CLI chatbot with all commands working
- Plan web interface (HTML, CSS, JS)
- Start building web_app.py HTTP server
- Design basic frontend UI layout

Progress/Achievements for the week:

- Complete CLI chatbot with all commands working
- Plan web interface (HTML, CSS, JS)
- Start building web_app.py HTTP server
- Design basic frontend UI layout

Future Work Plans:

- Build CSS styling for the web chat page
- Write JavaScript to connect frontend to backend API
- Test web chatbot in browser

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

Weekly project report for week commencing 30 march

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 30 March 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**

WPPR: 5

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

- Complete web frontend with CSS and JavaScript
- Connect frontend to /api/chat and /api/welcome endpoints
- Test full web chatbot in browser
- Fix any bugs found during testing

Progress/Achievements for the week:

- Built styles.css with responsive chat UI design
- Completed app.js with chat history management, quick action buttons and API calls
- Implemented /api/welcome endpoint that returns title, message and quick actions
- Successfully tested all chatbot commands in the browser
- Fixed JSON parsing errors and Unicode display issues in the terminal

Future Work Plans:

- Review overall project code quality
- Prepare High Level Design document
- Prepare Low Level Design document

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

Weekly project report for week commencing 6 April

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 6 April 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**

WPPR: 6

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

- Perform final end-to-end testing of both CLI and web versions
- Prepare project presentation
- Push all final files to GitHub

Progress/Achievements for the week:

- Completed High Level Design document with architecture diagram, process flow, information flow and API catalogue
- Completed Low Level Design document with module diagram, sequence diagram, data model and session flow
- Wrote full project report covering abstract, introduction, problem statement, objectives, system design, technology used, implementation, results and future scope
- Updated README.md with run instructions, command list and file descriptions

Future Work Plans:

- Perform final end-to-end testing of both CLI and web versions
- Prepare project presentation
- Push all final files to GitHub

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

Weekly project report for week commencing 13 April

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 13 April 2026

Programme: BCA (Artificial Intelligence,Cloud Computing and DevOps)

Student Name: Aswanth M O Register Number: **23BCAICD029**

WPPR: 7

Internal Guide's Name: Mr Praveen

MAJOR PROJECT Title:

Natural Language Interface for Agriculture Infrastructure

Targets set for the week:

- Perform final testing of CLI and web chatbot
- Fix any remaining bugs or issues
- Push all final code and documents to GitHub
- Prepare and finalise project presentation

Progress/Achievements for the week:

- Completed final end-to-end testing of CLI chatbot with all 10 commands verified
- Completed final testing of web chatbot in browser including quick action buttons
- Fixed minor Unicode encoding issue in print_console function in utils.py
- Pushed all final Python files, web files, HLD, LLD and project report to GitHub
- Prepared project presentation slides covering objectives, architecture, demo and future scope
- Project successfully completed and ready for submission

Future Work Plans:

- Future enhancements: integrate real-time weather API
- Future enhancements: implement machine learning for better NLP
- Future enhancements: develop mobile application version

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**